

3. Technologie i metodyki wykorzystane w projekcie

W rozdziale opisano dobór technologii oraz metodyk projektowych zastosowanych przy budowie platformy MoodFlow. Każda decyzja techniczna została skonfrontowana z trzema kryteriami: spełnieniem wymagań projektu (rozdziały 2.2 i 2.3), adekwatnością do skali projektu inżynierskiego oraz możliwością merytorycznego uzasadnienia w pracy. Pominęto technologie eksperymentalne i takie, których wybór byłby trudny do obrony przed komisją.

3.1. Założenia metodyczne

U podstaw projektu leży kilka założeń metodycznych, które wpłynęły na całą architekturę systemu.

Architektura modularna w stylu DDD-light

Backend zorganizowano w niezależne moduły domenowe (*auth*, *users*, *organizations*, *assessments*, *results*, *analytics*, *notifications*, *audit*, *health*), z których każdy zawiera własny kontroler, serwis, repozytoria oraz definicje encji. Podejście to wynika z metodyki DDD (Domain-Driven Design) w jej uproszczonej formie i pozwala na czytelne mapowanie kodu na obszary biznesowe systemu.

Wzorzec architektury wielodostępnej (multi-tenant)

Zastosowano model *row-level multi-tenancy* — wszystkie dane wszystkich firm znajdują się w jednej bazie danych, a izolacja jest realizowana przez filtrowanie zapytań po identyfikatorze organizacji pobieranym z tokenu uwierzytelniającego. Alternatywne modele (osobna baza per firma, osobny schemat) odrzucono jako zbyt kosztowne operacyjnie dla tej skali projektu.

Priorytetyzacja wymagań metodyką MoSCoW

Wymagania funkcjonalne i нефункционалне zostały skategoryzowane zgodnie z metodyką MoSCoW (rozdziały 2.1–2.4), co umożliwiło świadome rozdzielenie funkcjonalności krytycznych od pomocniczych oraz uzasadnienie elementów świadomie wykluczonych z bieżącej wersji.

Anonimizacja danych metodą k-anonimity

Dla raportów dla działu HR przyjęto regułę *k-anonimity* z progiem $k=5$ — agregaty są ukrywane dla grup mniejszych niż pięć osób. Metoda ta jest uznanym standardem w przetwarzaniu danych medycznych i pozwala zachować równowagę pomiędzy użytecznością analityczną a ochroną prywatności.

3.2. Język programowania — TypeScript

Cały kod aplikacji — zarówno backend, jak i oba frontendy — został napisany w języku **TypeScript**. Jest to nadbudowa nad JavaScriptem rozwijana przez firmę Microsoft, dodająca do języka system typów statycznych weryfikowanych podczas kompilacji.

Wybór TypeScriptu wynikał z następujących przesłanek:

- **Bezpieczeństwo typowe** — znaczna część błędów programistycznych (np. próba odczytu właściwości z `undefined`) jest wykrywana na etapie kompilacji, a nie w działającej aplikacji. W kontekście danych dotyczących zdrowia psychicznego minimalizacja ryzyka błędów ma znaczenie krytyczne.
- **Spójność typów end-to-end** — ten sam typ użytkownika lub wyniku testu może być współdzielony pomiędzy backendem a frontendem, co eliminuje rozbieżności w kontraktach API.
- **Wsparcie nowoczesnych frameworków** — NestJS, Next.js, Vite oraz Redux Toolkit traktują TypeScript jako pierwszorzędny język, co przekłada się na lepszą jakość dokumentacji i ekosystemu bibliotek.
- **Wartość edukacyjna i zawodowa** — TypeScript jest standardem branżowym w nowoczesnym *web developmencie*, a praca napisana w tym języku łatwiej spełnia wymagania utrzymania w przyszłości.

3.3. Backend — Node.js i NestJS

Warstwa serwerowa aplikacji została zbudowana w środowisku **Node.js 20** z użyciem frameworka **NestJS** w wersji 11. NestJS wprowadza do ekosystemu JavaScript koncepcje znane z dojrzałych frameworków takich jak Spring (Java) lub Angular — modułowość, wstrzykiwanie zależności (*dependency injection*), dekoratory oraz wyraźny podział na warstwy.

Uzasadnienie wyboru NestJS:

- **Modularna struktura kodu** — każdy obszar domenowy aplikacji (np. testy, użytkownicy, audyt) jest niezależnym modulem z jasno określonymi zależnościami, co odpowiada wymogowi czytelnej architektury opisanemu w pracy inżynierskiej.
- **Wbudowane mechanizmy bezpieczeństwa** — system *Guards* umożliwia deklaratywne zabezpieczenie endpointów rolami użytkownika, a globalny `ValidationPipe` automatycznie waliduje dane wejściowe na podstawie definicji DTO.
- **Pierwszorzędne wsparcie TypeScript** — framework wykorzystuje dekoratory i metadata reflection, dzięki czemu typowanie kodu jest ściśle i czytelne.

- **Bogaty ekosystem oficjalnych modułów** — `@nestjs/jwt`, `@nestjs/passport`, `@nestjs/typeorm`, `@nestjs/throttler` pozwoliły na szybkie wdrożenie kluczowych funkcji bez poszukiwania niesprawdzonych bibliotek zewnętrznych.

Alternatywą rozważaną był framework Express (minimalistyczny). Został odrzucony, ponieważ wymagałby ręcznego budowania wszystkich abstrakcji (modularność, walidacja, autoryzacja), co rozszerzyłoby zakres pracy bez wartości merytorycznej. Z kolei Fastify, choć technicznie wydajniejszy, ma uboższy ekosystem i mniejszą bazę dokumentacji w języku polskim.

3.4. Trwałość danych — PostgreSQL i TypeORM

Jako system zarządzania bazą danych wybrano **PostgreSQL 16** — relacyjną bazę danych z otwartym kodem źródłowym. Komunikacja z bazą odbywa się poprzez **TypeORM** — bibliotekę typu ORM (Object-Relational Mapping) ściśle zintegrowaną z NestJS.

Uzasadnienie wyboru PostgreSQL:

- **Natywne wsparcie dla typu JSONB** — wykorzystywanego przez tabele `audit_logs` (metadane akcji) oraz `assessment_results` (snapshot odpowiedzi). Wsparcie indeksów GIN umożliwia efektywne zapytania po polach JSON.
- **Pełniejsza zgodność ze standardem SQL** — window functions, CTE (Common Table Expressions), typy ENUM i CHECK constraints są przydatne w analityce HR.
- **Dojrzałość i stabilność** — PostgreSQL ma ponad 30-letnią historię rozwoju i jest obsługiwany przez wszystkich znaczących dostawców chmurowych (AWS RDS, Google Cloud SQL, Azure Database, Railway, Neon, Supabase).
- **Otwarta licencja** — brak kosztów licencyjnych zarówno na etapie projektu, jak i przy ewentualnym wdrożeniu komercyjnym.

Alternatywą rozważaną był MySQL (sugerowany w pierwotnym PRD), jednak po analizie potrzeb projektu (typ JSONB, audit log, anonimizacja agregatów) zdecydowano o zmianie na PostgreSQL. Decyzja ta została udokumentowana jako jedna z architektonicznych (ADR-002).

TypeORM wybrano zamiast alternatyw (Prisma, Sequelize, Knex.js) z uwagi na pełną integrację z NestJS, wsparcie dla dekoratorów typescriptowych oraz natywne mechanizmy migracji.

3.5. Frontend pracownika — React i Vite

Aplikacja przeznaczona dla pracownika została zrealizowana w bibliotece **React 18** z wykorzystaniem narzędzia budującego **Vite 8**. Stan aplikacji zarządzany jest przez **Redux Toolkit**, a routing realizuje biblioteka **React Router**.

Uzasadnienie:

- **React** — najczęściej stosowana biblioteka do budowy interfejsów webowych, z bogatym ekosystemem komponentów oraz dużą społecznością. Komponentowy model programowania ułatwia ponowne wykorzystanie elementów (np. `StatCard`, `ClientCard`) i jest zgodny z zasadą rozdzielania odpowiedzialności.
- **Vite** — nowoczesne narzędzie budujące zastępujące starsze rozwiązania (Webpack, Create React App). Czas startu serwera deweloperskiego w typowym projekcie wynosi poniżej sekundy, a hot module replacement działa niemal natychmiastowo. Vite oferuje również wsparcie dla budowania PWA poprzez `vite-plugin-pwa`.
- **Redux Toolkit** — oficjalna, opinionated wersja Reduksa eliminująca dawne problemy boilerplate'u. Zarządza stanem uwierzytelnienia oraz wynikami zapytań do API. Dla aplikacji średniej skali Redux Toolkit zapewnia balans pomiędzy strukturą a prostotą.
- **React Router 6** — standard de facto dla routingu w aplikacjach React.

Alternatywami rozważanymi były Vue.js i Svelte. React wybrano z uwagi na rozmiar ekosystemu, wsparcie dla TypeScriptu oraz wartość edukacyjną — jest to najbardziej rozpoznawana biblioteka w polskich ofertach pracy w 2025 roku.

3.6. Frontend administracyjny — Next.js

Panele administracyjne (administrator firmy, HR, właściciel platformy) zostały zbudowane przy użyciu frameworka **Next.js 16** — nadbudowy nad Reactem rozwijanej przez firmę Vercel. Next.js wprowadza renderowanie po stronie serwera (SSR), automatyczne dzielenie kodu (*code splitting*), routing oparty o strukturę folderów oraz model *App Router* z wsparciem dla React Server Components.

Uzasadnienie wyboru:

- **SEO i wydajność wstępnego renderu** — choć panele administracyjne nie są publiczne, SSR przyspiesza pierwszy render i poprawia odbiór aplikacji.
- **Routing oparty o pliki** — struktura folderów `app/` bezpośrednio odzwierciedla strukturę URL, co jest szczególnie przydatne przy wielu zagnieżdżonych panelach (administracyjnym, HR, super-admina).

- **Wsparcie dla TypeScriptu** oraz integracja z bibliotekami React (Recharts, Framer Motion).
- **Świadome rozdzielenie aplikacji pracownika i administracyjnej** — każda ma inne wymagania (PWA dla pracownika, bogata analityka dla admina), więc wybór różnych frameworków pozwolił na optymalizację pod konkretny przypadek użycia.

3.7. Stylowanie i komponenty UI — Tailwind CSS

Wszystkie warstwy prezentacji aplikacji wykorzystują **Tailwind CSS** — framework typu *utility-first*, w którym style są aplikowane bezpośrednio jako klasy w kodzie HTML/JSX. W projekcie zastosowano wersję 4.

Uzasadnienie:

- **Spójność wizualna** — predefiniowane skale kolorów, marginesów i typografii eliminują niespójności wynikające z ręcznego pisania CSS.
- **Mała wielkość finalnej paczki** — Tailwind generuje wyłącznie te klasy, które zostały faktycznie użyte w kodzie, dzięki czemu rozmiar arkusza stylów w produkcji jest minimalny.
- **Łatwość modyfikacji motywu** — system tokenów CSS (`--bg-page`, `--text-base`, itd.) pozwolił na zaimplementowanie trybu jasnego i ciemnego z minimalną ingerencją w komponenty.
- **Mobile-first** — wbudowane modyfikatory `sm:`, `md:`, `lg:` wymuszają projektowanie zaczynając od urządzeń mobilnych, co odpowiada wymogom NF14.

Alternatywą rozważaną był system styled-components (CSS-in-JS), odrzucony z uwagi na większy *runtime overhead* oraz mniejsze wsparcie dla statycznej analizy.

3.8. Wizualizacja danych — Recharts oraz SVG

W panelu HR oraz na pulpicie pracownika prezentowane są wykresy: trendy w czasie, rozkłady wyników, indeks dobrostanu. Wykorzystano dwa podejścia:

- **Recharts** — biblioteka komponentów wykresów dla Reacta, oparta na D3.js. Wykorzystywana w panelu HR (Bar Chart aktywności, Line Chart trendów, Pie Chart rozkładu nasilenia objawów). Recharts oferuje wbudowaną responsywność oraz spójną stylistykę.
- **Wykresy SVG własnego autorstwa** — w aplikacji pracownika wykresy indeksu dobrostanu i historii wyników zostały zaimplementowane bez zewnętrznej biblioteki, z

użyciem natywnych elementów SVG. Pozwala to na precyzyjną kontrolę nad wyglądem (krzywa Beziera dla wygładzonej linii, gradient pod krzywą, animowane wartości).

Recharts wybrano nad alternatywami (Chart.js, ApexCharts) z uwagi na natywną integrację z React (komponenty zamiast imperatywnego API canvas) oraz lekki rozmiar.

3.9. Progressive Web App

Aplikacja pracownika została zaimplementowana jako **Progressive Web App (PWA)** z wykorzystaniem wtyczki `vite-plugin-pwa` opartej na bibliotece **Workbox** (Google).

Implementacja zawiera:

- **Manifest aplikacji** (`manifest.webmanifest`) — opisuje nazwę, opis, kolor motywu, ikony oraz tryb wyświetlania (*standalone*);
- **Ikony w formatach PNG** dla iOS i Androida (192×192, 512×512, 180×180 dla *apple-touch-icon*);
- **Service Worker** — buforuje zasoby statyczne (kod JavaScript, arkusze stylów, czcionki) z polityką *cache-first*, dzięki czemu kolejne uruchomienia są niemal natychmiastowe;
- **Strategie wykluczenia cache** — endpointy autoryzacji (`/auth/*`, `/users/me`, `/results`) są wykluczone z buforowania, aby zapobiec wyciekowi danych po wylogowaniu;
- **Mechanizm powiadamiania o nowych wersjach** — tryb `registerType: 'prompt'` pokazuje użytkownikowi banner z propozycją odświeżenia, dzięki czemu nigdy nie traci on niezapisanych danych formularza.

Wybór PWA jako alternatywy dla aplikacji natywnych iOS/Android pozwolił zachować pojedynczą bazę kodu i wyeliminować konieczność dystrybucji przez sklepy Apple App Store i Google Play.

3.10. Bezpieczeństwo aplikacji

W warstwie bezpieczeństwa wykorzystano zestaw sprawdzonych bibliotek branżowych:

- **bcrypt** — hashowanie haseł z 12 rundami soli;
- **JSON Web Tokens (JWT)** — tokeny dostępu (15 minut) oraz odświeżania (7 dni), podpisywane algorytmem HS256 z różnymi sekretami;
- **Helmet** — automatyczne dołączanie nagłówków bezpieczeństwa HTTP (X-Frame-Options, X-Content-Type-Options, HSTS, Referrer-Policy);

- **cookie-parser** — parsowanie ciasteczek HttpOnly z tokenami JWT;
- **class-validator** i **class-transformer** — walidacja danych wejściowych w DTO z deklaratywnymi dekoratorami (`@IsEmail`, `@MinLength`, `@IsString`);
- **@nestjs/throttler** — ograniczenie liczby żądań na endpointy uwierzytelniające (login: 10 prób na 15 minut, rejestracja: 5 prób na 15 minut).

Szczegółowy opis każdego z mechanizmów bezpieczeństwa został zawarty w rozdziale poświęconym bezpieczeństwu aplikacji.

3.11. Konteneryzacja i wdrożenie

Docker oraz Docker Compose

Wszystkie komponenty aplikacji (backend, dwa frontendy, baza danych) działają w izolowanych kontenerach **Docker**. Orkiestrację realizuje narzędzie **Docker Compose** w dwóch konfiguracjach: `docker-compose.yml` dla środowiska rozwojowego (z *hot reload*, narzędziem Adminer dla bazy danych, bind-mount wolumenów źródłowych) oraz `docker-compose.prod.yml` dla środowiska produkcyjnego (z *health checkami*, polityką restartu `unless-stopped`, bazą danych dostępną tylko wewnętrznie).

Konteneryzacja zapewnia identyczne środowisko uruchomieniowe na różnych platformach (lokalnie, na chmurze, w CI/CD) i jest standardem branżowym w nowoczesnych aplikacjach webowych.

Platforma chmurowa Railway

Aplikacja produkcyjna została wdrożona na platformie **Railway** — nowoczesnym dostawcy chmurowym typu PaaS (Platform as a Service), zorientowanym na deweloperów. Railway integruje się z repozytorium GitHub i automatycznie wdraża zmiany po każdym *push*'u do gałęzi produkcyjnej.

Wybór Railway:

- Natywne wsparcie dla aplikacji w kontenerach Docker oraz dla plików `docker-compose.yml`;
- Automatyczne udostępnianie certyfikatów TLS (Let's Encrypt) dla każdej domeny;
- Managed PostgreSQL z dziennymi snapshotami i wewnętrznym DNS (`postgres.railway.internal`) niedostępnym z internetu;
- Wbudowane narzędzia obserwowalności (logi, metryki) i healthchecki;

- Hobby tier z 5 USD kredytów miesięcznie wystarczający na potrzeby projektu inżynierskiego.

Alternatywami rozważanymi były Render, Fly.io oraz wirtualny serwer prywatny (VPS) z konfiguracją od zera. Railway wybrano z uwagi na najprostszy proces wdrażania monorepo z wieloma serwisami oraz dobrą dokumentację.

3.12. Narzędzia wspierające proces tworzenia

Git i GitHub

Cały kod projektu utrzymywany jest w systemie kontroli wersji **Git**, w repozytorium hostowanym na **GitHub**. Każda istotna zmiana jest dokumentowana czytelnym komunikatem *commit*, a krytyczne decyzje techniczne wpisywane są do dziennika decyzji architektonicznych w katalogu `docs/`.

Visual Studio Code

Środowiskiem rozwojowym był **Visual Studio Code** z dodatkami: ESLint, Prettier, TypeScript Vue Plugin, Tailwind CSS IntelliSense, GitLens.

Mermaid — diagramy w formacie tekstowym

Wszystkie diagramy projektowe (use case, klas, sekwencji, ERD, komponentów, wdrożenia, BPMN) zostały opisane składnią **Mermaid** — językiem do generowania diagramów z opisu tekstowego. Mermaid jest natywnie renderowany w plikach Markdown na GitHub i większości nowoczesnych edytorów. Dzięki temu diagramy są wersjonowane razem z kodem i można je modyfikować w sposób kontrolowany przez *code review*.

Postman / Thunder Client

Testy ręczne API w trakcie rozwoju realizowane były przy użyciu klienta REST (**Postman** oraz wbudowany **Thunder Client** w VS Code).

Pencil i Figma — projektowanie interfejsu

Wstępne makiety interfejsu zostały przygotowane w narzędziach do projektowania UI — **Pencil** (formaty `.pen`) oraz dla wybranych ekranów w **Figma**. Stanowiły one bazę referencyjną dla finalnej implementacji w Tailwind CSS.

Podsumowanie doboru technologii

Stos technologiczny projektu MoodFlow został zaprojektowany pod kątem trzech celów:

1. **Spełnienia wymagań projektu** — każda technologia odpowiada konkretnym wymaganiom funkcjonalnym lub нефunkcjonalnym opisanym w rozdziale 2.
2. **Adekwatności do skali projektu inżynierskiego** — preferowano rozwiązania popularne, dobrze udokumentowane, możliwe do obrony na obronie pracy bez konieczności tłumaczenia eksperymentalnych decyzji.
3. **Aktualności rynkowej** — wszystkie wybrane technologie są standardem branżowym w 2025 roku, co zwiększa wartość edukacyjną i zawodową projektu.

Pełen stos technologiczny przedstawia się następująco: **TypeScript** jako język programowania, **NestJS + TypeORM + PostgreSQL** w warstwie backendowej, **React + Vite + Redux Toolkit** dla aplikacji pracownika, **Next.js** dla aplikacji administracyjnej, **Tailwind CSS** jako system stylowania, **Recharts** i **SVG** do wizualizacji danych, **vite-plugin-pwa + Workbox** dla funkcji PWA, **Docker + Docker Compose** dla konteneryzacji oraz **Railway** jako platforma chmurowa.